

Tarea 4

La actividad unifica la gestión de cursos y tareas académicas con un módulo de conversión de temperaturas, aplicando clases, métodos, validaciones, eventos y componentes gráficos.

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas.
Segundo Semestre 2026
Nombre: Evelyn Aracely Cardona Aguilón
Carnet: 202406816

No.	Tema / Concepto a investigar	Referencia sugerida
1	Programación Orientada a Objetos: POO: entidades centrales son los objetos Objetos: es una colección de datos llamado variables y las subrutinas o métodos que operan sobre ellos. pueden existir unos métodos especiales denominados constructores que se llaman siempre que se crea un objeto de esa clase y cuya misión es iniciar el objeto. Objeto: Es una instancia de una clase, es una colección de datos llamado variables y las subrutinas o métodos que operan sobre ellos. Atributo: variable que define las características, propiedades o estado de una clase o de un objeto en la Programación Orientada a Objetos Método: comportamientos en objetos	LIBRO JAVA
2	Los cuatro pilares de la POO Encapsulamiento: Es la clase donde se define la estructura y el comportamiento Herencia: En programación orientada a objetos, la reutilización de código se efectúa creando una subclase que constituye una restricción o extensión de la clase base, de la cual hereda sus propiedades. polimorfismo: Va de la mano con la herencia poli = muchas morfismo = formas a partir de un objeto que pertenece a una madre puedo tener clases hijas se consideran a un solo objeto aunque tenga muchos hijos Abstracción: consiste en aislar los elementos esenciales de un objeto para crear un molde (clase), ignorando los detalles complejos o de relleno que no son necesarios para el sistema.	GeeksforGeeks - POO in Java
3	Diseño de clases para representar cursos y tareas académicas.	Material del laboratorio
4	Java Swing: JFrame, JPanel, JLabel, JTextField, JButton, JTextArea y JOptionPane. Java Swing es una biblioteca integrada en Java que sirve para crear Interfaces Gráficas de Usuario (GUI). En lugar de ejecutar tus programas en una pantalla negra de consola, Swing te permite diseñar ventanas con botones, cajas de texto, imágenes y menús.	Java Swing Tutorial

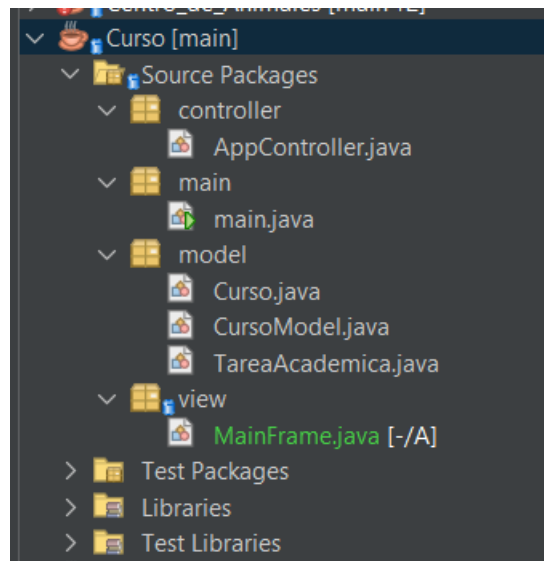
	JFRAME: ventana principal, es el contenedor base que sostiene a todos los elementos tiene los botones clasicos como cerrar, maximizar y minimiza JPANEL: Organizador de Espacio, se mete dentro de un JFRAME base sirve para agrupar, ordenar y organizar los botones y textos en diferentes secciones de la pantalla evitando que quede todo junto. JLABEL: Etiquetas de textos o imágenes, Sirve para mostrar texto estático en la pantalla que el usuario no puede editar. Se usa principalmente para poner títulos, instrucciones o nombres al lado de los campos JTEXTFIELD: Caja de texto corto, Es un campo o cuadro blanco de una sola línea donde el usuario puede escribir datos desde su teclado. Se usa para pedir datos cortos como un nombre etc. JBUTTON: El boton de accion, Es un botón clásico sobre el que el usuario puede hacer clic. Al presionarlo, el programa reacciona y ejecuta una acción. JTEXTAREA: caja de texto Largo. JOPTIONPANE: Ventanas Emergentes/Alertas, mensaje de aviso flotantes como contraseña incorrecta.	
5	Manejo de eventos en Java: ActionListener y respuesta a botones: Manejo de Eventos es el mecanismo que hace que tu interfaz gráfica "cobre vida" Por defecto, cuando dibujas un JButton en la pantalla y haces clic sobre él, no pasa absolutamente nada. El botón se hunde visualmente, pero está "muerto". Para que al hacer clic se ejecute tu código de Java, necesitas dos cosas: un Evento (el clic) y un Escuchador (el ActionListener).	Oracle Java Tutorials
6	Validación de entradas numéricas y textuales en aplicaciones Java. La validación de entradas es el proceso de revisar que lo que el usuario escriba en tu aplicación de Java sea correcto antes de que tu programa intente procesarlo.	Apuntes de clase / W3Schools Java

Sección 2: Explicación del Diseño de Clases (Arquitectura)

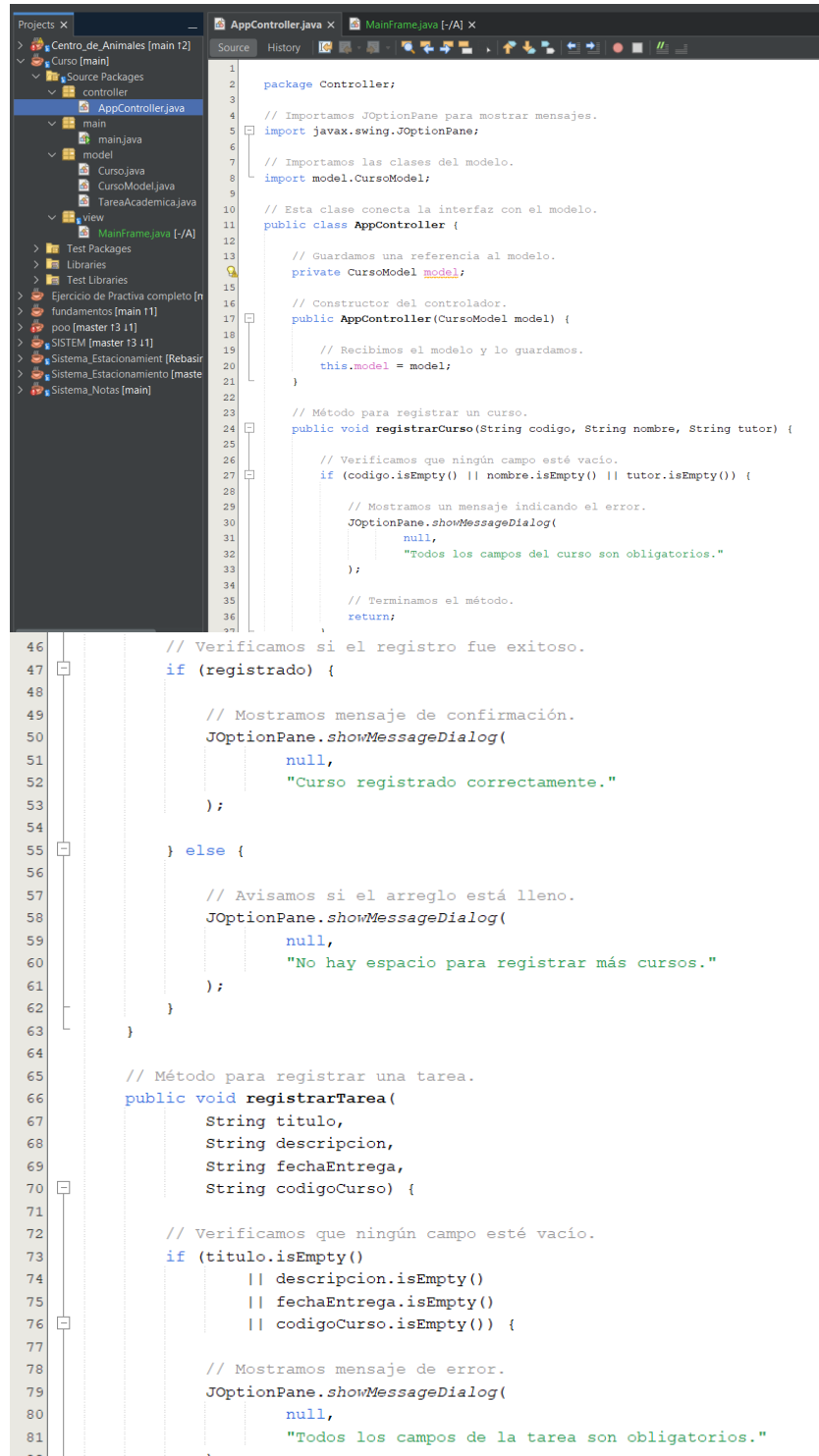
- Model (Curso, TareaAcademica, CursoModel): Representa la estructura de datos y los arreglos primitivos.
- View (MainFrame): Contiene la interfaz gráfica construida manualmente con componentes Swing sin GUI Builder.
- Controller (AppController): Gestiona la lógica entre los formularios y las validaciones.

Sección 3: Imágenes de mi IDE códigos de mi Tarea

1.1 Imagen de la estructura que utilice es la de CMV en cada uno cree sus respectivas clases.



1.2 PAQUETE CONTROLLER CLASE AppController



```
1 package Controller;
2
3 // Importamos JOptionPane para mostrar mensajes.
4 import javax.swing.JOptionPane;
5
6 // Importamos las clases del modelo.
7 import model.CursoModel;
8
9 // Esta clase conecta la interfaz con el modelo.
10 public class AppController {
11
12     // Guardamos una referencia al modelo.
13     private CursoModel model;
14
15     // Constructor del controlador.
16     public AppController(CursoModel model) {
17
18         // Recibimos el modelo y lo guardamos.
19         this.model = model;
20     }
21
22     // Método para registrar un curso.
23     public void registrarCurso(String codigo, String nombre, String tutor) {
24
25         // Verificamos que ningún campo esté vacío.
26         if (codigo.isEmpty() || nombre.isEmpty() || tutor.isEmpty()) {
27
28             // Mostramos un mensaje indicando el error.
29             JOptionPane.showMessageDialog(
30                 null,
31                 "Todos los campos del curso son obligatorios."
32             );
33
34             // Terminamos el método.
35             return;
36
37         // Verificamos si el registro fue exitoso.
38         if (registrado) {
39
40             // Mostramos mensaje de confirmación.
41             JOptionPane.showMessageDialog(
42                 null,
43                 "Curso registrado correctamente."
44             );
45
46         } else {
47
48             // Avisamos si el arreglo está lleno.
49             JOptionPane.showMessageDialog(
50                 null,
51                 "No hay espacio para registrar más cursos."
52             );
53
54         }
55     }
56
57     // Método para registrar una tarea.
58     public void registrarTarea(
59         String titulo,
60         String descripcion,
61         String fechaEntrega,
62         String codigoCurso) {
63
64         // Verificamos que ningún campo esté vacío.
65         if (titulo.isEmpty()
66             || descripcion.isEmpty()
67             || fechaEntrega.isEmpty()
68             || codigoCurso.isEmpty()) {
69
70             // Mostramos mensaje de error.
71             JOptionPane.showMessageDialog(
72                 null,
73                 "Todos los campos de la tarea son obligatorios."
74             );
75
76         }
77     }
78 }
```

```

35         // Terminamos el método.
36         return;
37     }
38
39     // Enviamos los datos directamente al modelo.
40     boolean registrado = model.registrarCurso(
41         codigo,
42         nombre,
43         tutor
44     );
45
46     // Verificamos si el registro fue exitoso.
47     if (registrado) {
48
49         // Mostramos mensaje de confirmación.
50         JOptionPane.showMessageDialog(
51             null,
52             "Curso registrado correctamente."
53         );
54
55     } else {
56
57         // Avisamos si el arreglo está lleno.
58         JOptionPane.showMessageDialog(
59             null,
60             "No hay espacio para registrar más cursos."
61         );
62     }
63 }
64
65 // Método para registrar una tarea.
66 public void registrarTarea(
67     String titulo,
68     String descripcion,
69     String fechaEntrega,
70     String codigoCurso) {
71
72     // Verificamos que ningún campo esté vacío.
73     if (titulo.isEmpty()
74         || descripcion.isEmpty()
75         || fechaEntrega.isEmpty()
76         || codigoCurso.isEmpty()) {
77
78         // Mostramos mensaje de error.
79         JOptionPane.showMessageDialog(
80             null,
81             "Todos los campos de la tarea son obligatorios."
82         );
83
84         // Terminamos el método.
85         return;
86     }
87
88     // Verificamos que el curso indicado exista.
89     boolean cursoExiste = false;
90
91     // Recorremos los cursos registrados.
92     for (int i = 0; i < model.getCantidadCursos(); i++) {
93
94         // Comparamos el código escrito con el código del curso.
95         if (model.getCursos()[i].getCodigo().equals(codigoCurso)) {
96
97             // Encontramos el curso.
98             cursoExiste = true;
99
100             // Ya no necesitamos seguir buscando.
101             break;
102         }
103     }
104
105     // Si el curso no existe, no registramos la tarea.
106     if (!cursoExiste) {
107

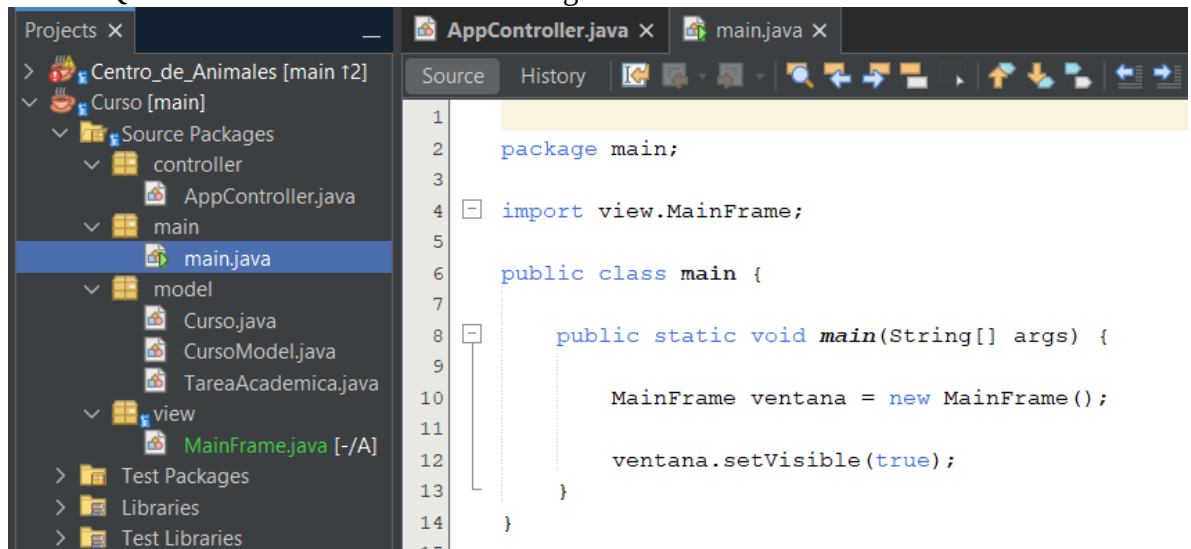
```

```

107
108 // Mostramos el mensaje de error.
109 JOptionPane.showMessageDialog(
110     null,
111     "El código del curso no existe."
112 );
113
114 // Terminamos el método.
115 return;
116
117
118 // Enviamos los datos directamente al modelo.
119 boolean registrada = model.registrarTarea(
120     titulo,
121     descripcion,
122     fechaEntrega,
123     codigoCurso
124 );
125
126 // Verificamos si se pudo guardar.
127 if (registrada) {
128
129     // Mostramos mensaje de confirmación.
130     JOptionPane.showMessageDialog(
131         null,
132         "Tarea registrada correctamente."
133     );
134
135 } else {
136
137     // Avisamos si el arreglo está lleno.
138     JOptionPane.showMessageDialog(
139         null,
140         "No hay espacio para registrar más tareas."
141     );
142
143 }

```

1.3 PAQUETE Main Y CLASE MAIN original



The screenshot shows an IDE with a project named 'Centro_de_Animales [main t2]'. The project structure is as follows:

- Centro_de_Animales [main t2]
 - Curso [main]
 - Source Packages
 - controller
 - AppController.java
 - main
 - main.java (selected)
 - model
 - Curso.java
 - CursoModel.java
 - TareaAcademica.java
 - view
 - MainFrame.java [-/A]
 - Test Packages
 - Libraries
 - Test Libraries

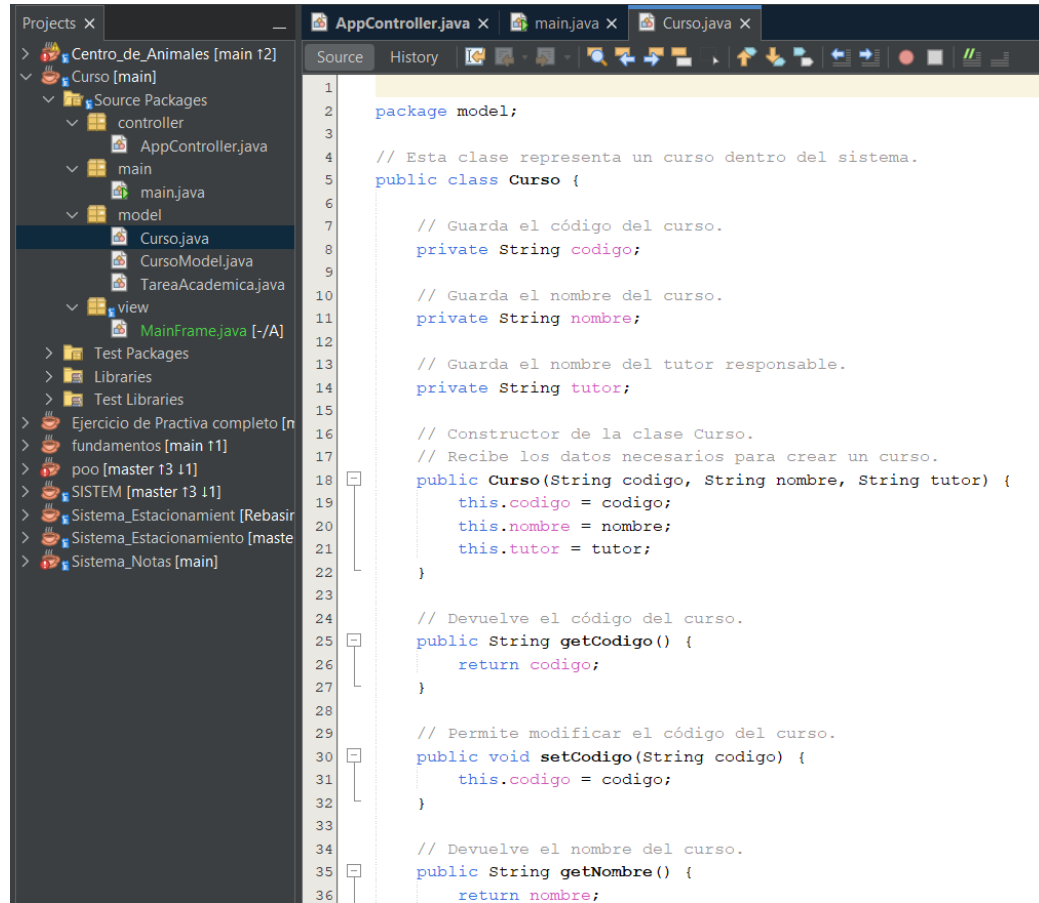
The 'main.java' file is open in the editor, showing the following code:

```

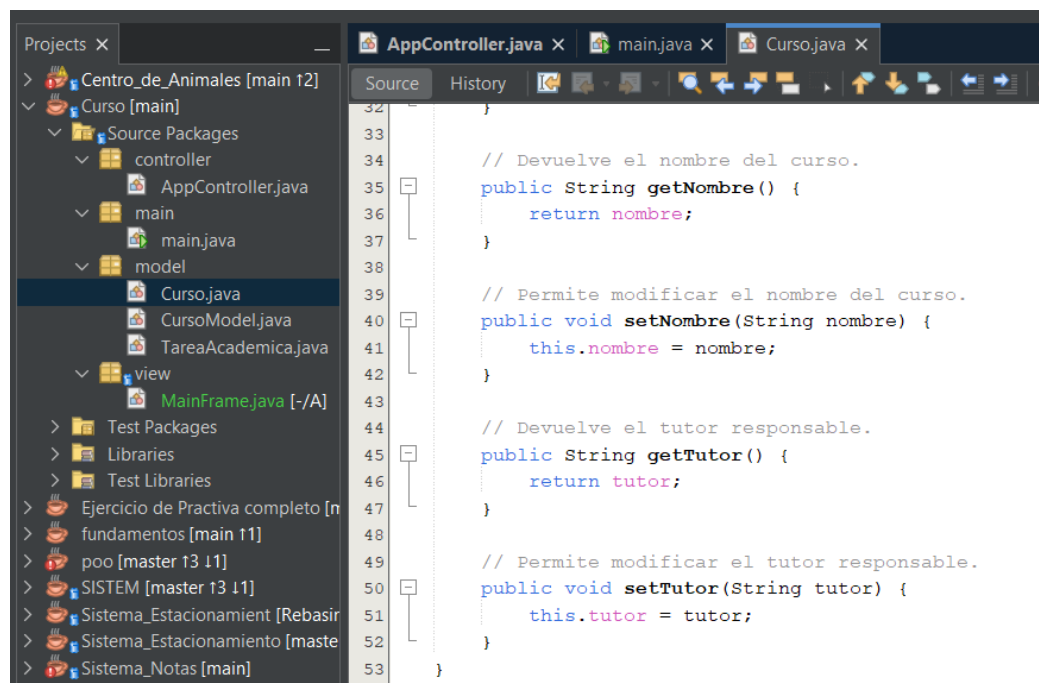
1 package main;
2
3
4 import view.MainFrame;
5
6 public class main {
7
8     public static void main(String[] args) {
9
10         MainFrame ventana = new MainFrame();
11
12         ventana.setVisible(true);
13     }
14 }

```

1.4 PAQUETE MODEL CLASE CURSO

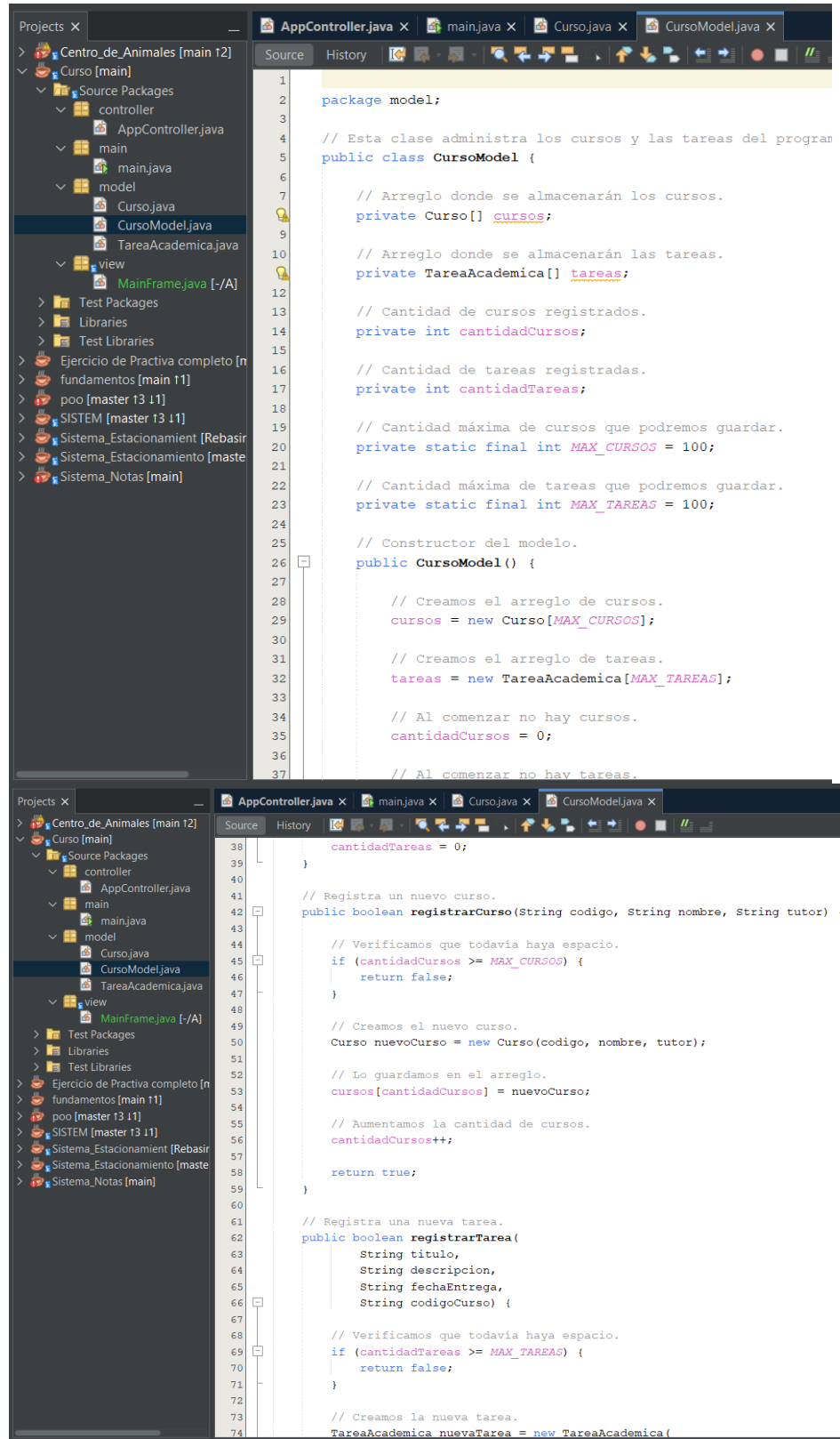


```
1 package model;
2
3 // Esta clase representa un curso dentro del sistema.
4 public class Curso {
5
6     // Guarda el código del curso.
7     private String codigo;
8
9     // Guarda el nombre del curso.
10    private String nombre;
11
12    // Guarda el nombre del tutor responsable.
13    private String tutor;
14
15    // Constructor de la clase Curso.
16    // Recibe los datos necesarios para crear un curso.
17    public Curso(String codigo, String nombre, String tutor) {
18        this.codigo = codigo;
19        this.nombre = nombre;
20        this.tutor = tutor;
21    }
22
23    // Devuelve el código del curso.
24    public String getCodigo() {
25        return codigo;
26    }
27
28    // Permite modificar el código del curso.
29    public void setCodigo(String codigo) {
30        this.codigo = codigo;
31    }
32
33    // Devuelve el nombre del curso.
34    public String getNombre() {
35        return nombre;
36    }
```



```
32 }
33
34 // Devuelve el nombre del curso.
35 public String getNombre() {
36     return nombre;
37 }
38
39 // Permite modificar el nombre del curso.
40 public void setNombre(String nombre) {
41     this.nombre = nombre;
42 }
43
44 // Devuelve el tutor responsable.
45 public String getTutor() {
46     return tutor;
47 }
48
49 // Permite modificar el tutor responsable.
50 public void setTutor(String tutor) {
51     this.tutor = tutor;
52 }
53 }
```

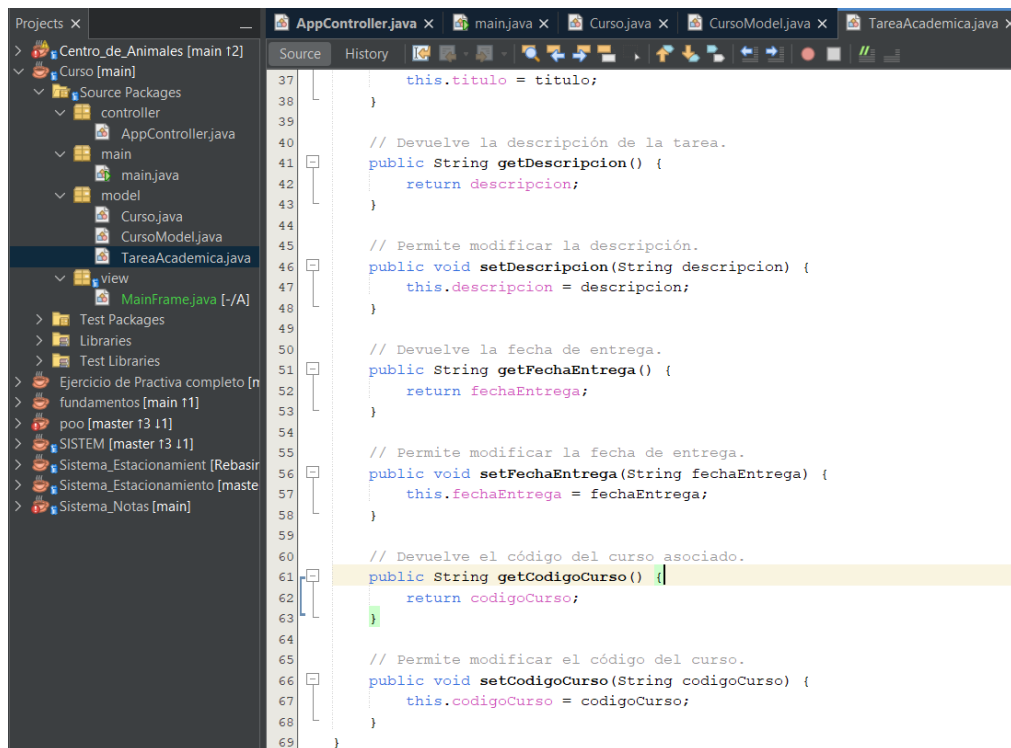
1.4 CLASE CURSO MODEL



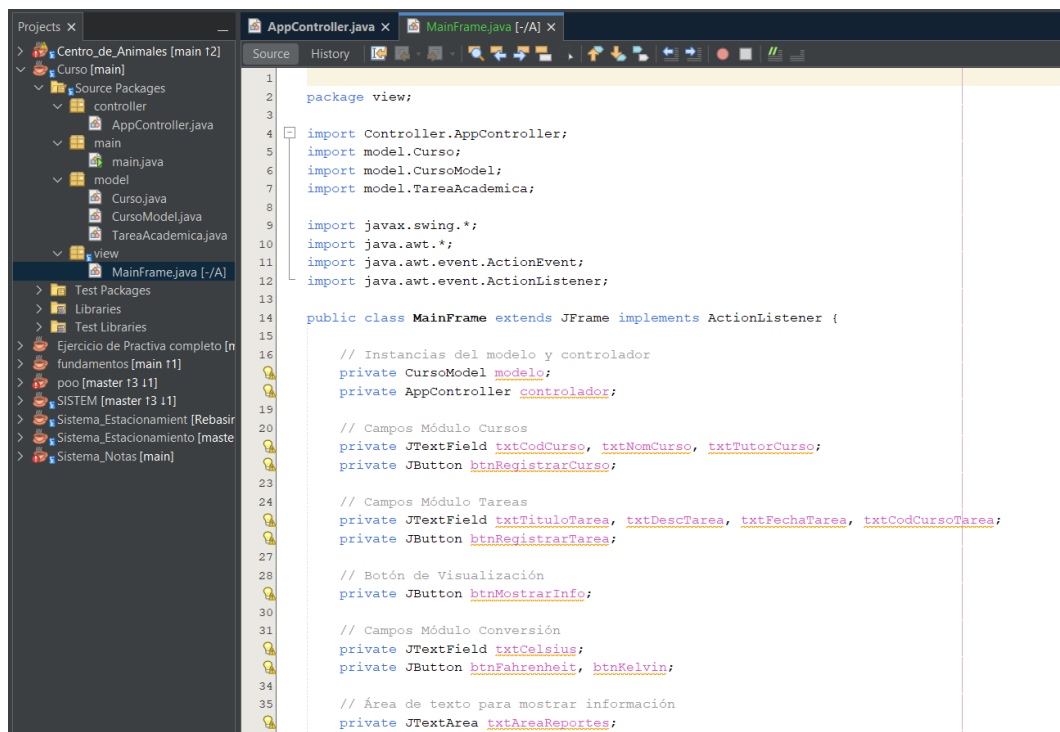
```
74 TareaAcademica nuevaTarea = new TareaAcademica(  
75     titulo,  
76     descripcion,  
77     fechaEntrega,  
78     codigoCurso  
79 );  
80  
81 // La guardamos en el arreglo.  
82 tareas[cantidadTareas] = nuevaTarea;  
83  
84 // Aumentamos la cantidad de tareas.  
85 cantidadTareas++;  
86  
87 return true;  
88 }  
89  
90 // Devuelve el arreglo de cursos.  
91 public Curso[] getCursos() {  
92     return cursos;  
93 }  
94  
95 // Devuelve la cantidad de cursos registrados.  
96 public int getCantidadCursos() {  
97     return cantidadCursos;  
98 }  
99  
100 // Devuelve el arreglo de tareas.  
101 public TareaAcademica[] getTareas() {  
102     return tareas;  
103 }  
104  
105 // Devuelve la cantidad de tareas registradas.  
106 public int getCantidadTareas() {  
107     return cantidadTareas;  
108 }  
109 }
```

1.5 PAQUETE MODEL CLASE TAREAACADEMICA

```
1  
2 package model;  
3  
4 // Esta clase representa una tarea académica.  
5 public class TareaAcademica {  
6  
7     // Guarda el título de la tarea.  
8     private String titulo;  
9  
10    // Guarda la descripción de la tarea.  
11    private String descripcion;  
12  
13    // Guarda la fecha de entrega de la tarea.  
14    private String fechaEntrega;  
15  
16    // Guarda el código del curso al que pertenece la tarea.  
17    private String codigoCurso;  
18  
19    // Constructor de la clase TareaAcademica.  
20    // Recibe todos los datos necesarios para crear una tarea.  
21    public TareaAcademica(String titulo, String descripcion,  
22        String fechaEntrega, String codigoCurso) {  
23  
24        this.titulo = titulo;  
25        this.descripcion = descripcion;  
26        this.fechaEntrega = fechaEntrega;  
27        this.codigoCurso = codigoCurso;  
28    }  
29  
30    // Devuelve el título de la tarea.  
31    public String getTitulo() {  
32        return titulo;  
33    }  
34  
35    // Permite modificar el título.  
36    public void setTitulo(String titulo) {  
37        this.titulo = titulo;  
38    }  
39 }
```



1.6 PAQUETE VIEW CLASE MAINFRAME



```
Projects X
> Centro_de_Animales [main 12]
> Curso [main]
  > Source Packages
    > controller
      > AppController.java
    > main
      > main.java
    > model
      > Curso.java
      > CursoModel.java
      > TareaAcademica.java
    > view
      > MainFrame.java [-/A]
  > Test Packages
  > Libraries
  > Test Libraries
> Ejercicio de Práctica completo [n
> fundamentos [main 11]
> poo [master 13 11]
> SISTEM [master 13 11]
> Sistema_Estacionamient [Rebasir
> Sistema_Estacionamiento [maste
> Sistema_Notas [main]

AppController.java X MainFrame.java [-/A] X
Source History
37
38 public MainFrame() {
39     // Inicializar modelo y controlador
40     modelo = new CursoModel();
41     controlador = new AppController(modelo);
42
43     // Configuración de la ventana principal
44     setTitle("Gestión de Cursos, Tareas y Conversor");
45     setSize(850, 650);
46     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
47     setLocationRelativeTo(null);
48     setLayout(new BorderLayout(10, 10));
49
50     // --- PANEL SUPERIOR ---
51     JPanel panelSuperior = new JPanel(new GridLayout(1, 3, 10, 10));
52
53     // 1. Módulo Cursos
54     JPanel panelCursos = new JPanel(new GridLayout(4, 2, 5, 5));
55     panelCursos.setBorder(BorderFactory.createTitledBorder("Módulo de Cursos"));
56     panelCursos.add(new JLabel("Código:"));
57     txtCodCurso = new JTextField();
58     panelCursos.add(txtCodCurso);
59     panelCursos.add(new JLabel("Nombre:"));
60     txtNomCurso = new JTextField();
61     panelCursos.add(txtNomCurso);
62     panelCursos.add(new JLabel("Tutor:"));
63     txtTutorCurso = new JTextField();
64     panelCursos.add(txtTutorCurso);
65
66     btnRegistrarCurso = new JButton("Registrar Curso");
67     btnRegistrarCurso.addActionListener(this); // Asignación directa del evento
68     panelCursos.add(new JLabel(""));
69     panelCursos.add(btnRegistrarCurso);
70
71     // 2. Módulo Tareas
72     JPanel panelTareas = new JPanel(new GridLayout(5, 2, 5, 5));
```

```
Projects X
> Centro_de_Animales [main 12]
> Curso [main]
  > Source Packages
    > controller
      > AppController.java
    > main
      > main.java
    > model
      > Curso.java
      > CursoModel.java
      > TareaAcademica.java
    > view
      > MainFrame.java [-/A]
  > Test Packages
  > Libraries
  > Test Libraries
> Ejercicio de Práctica completo [n
> fundamentos [main 11]
> poo [master 13 11]
> SISTEM [master 13 11]
> Sistema_Estacionamient [Rebasir
> Sistema_Estacionamiento [maste
> Sistema_Notas [main]

AppController.java X MainFrame.java [-/A] X
Source History
73 panelTareas.setBorder(BorderFactory.createTitledBorder("Módulo de Tareas"));
74 panelTareas.add(new JLabel("Título:"));
75 txtTituloTarea = new JTextField();
76 panelTareas.add(txtTituloTarea);
77 panelTareas.add(new JLabel("Descripción:"));
78 txtDescTarea = new JTextField();
79 panelTareas.add(txtDescTarea);
80 panelTareas.add(new JLabel("Fecha entrega:"));
81 txtFechaTarea = new JTextField();
82 panelTareas.add(txtFechaTarea);
83 panelTareas.add(new JLabel("Código curso:"));
84 txtCodCursoTarea = new JTextField();
85 panelTareas.add(txtCodCursoTarea);
86
87 btnRegistrarTarea = new JButton("Registrar Tarea");
88 btnRegistrarTarea.addActionListener(this); // Asignación directa del evento
89 panelTareas.add(new JLabel(""));
90 panelTareas.add(btnRegistrarTarea);
91
92 // 3. Módulo de Visualización
93 JPanel panelMostrar = new JPanel(new GridBagLayout());
94 panelMostrar.setBorder(BorderFactory.createTitledBorder("Visualización"));
95 btnMostrarInfo = new JButton("Mostrar información");
96 btnMostrarInfo.setPreferredSize(new Dimension(160, 40));
97 btnMostrarInfo.addActionListener(this); // Asignación directa del evento
98 panelMostrar.add(btnMostrarInfo);
99
100 panelSuperior.add(panelCursos);
101 panelSuperior.add(panelTareas);
102 panelSuperior.add(panelMostrar);
103
104 add(panelSuperior, BorderLayout.NORTH);
105
106 // --- PANEL CENTRAL ---
107 txtAreaReportes = new JTextArea();
108 txtAreaReportes.setEditable(false);
```

```

Projects X
> Centro_de_Animales [main t2]
> Curso [main]
  > Source Packages
    > controller
      > AppController.java
    > main
      > main.java
    > model
      > Curso.java
      > CursoModel.java
      > TareaAcademica.java
    > view
      > MainFrame.java [-/A]
  > Test Packages
  > Libraries
  > Test Libraries
> Ejercicio de Practiva completo [n
> fundamentos [main t1]
> poo [master t3 t1]
> SISTEM [master t3 t1]
> Sistema_Estacionamient [Rebasir
> Sistema_Estacionamiento [maste
> Sistema_Notas [main]

AppController.java X MainFrame.java [-/A] X
Source History
109 JScrollPane scrollArea = new JScrollPane(txtAreaReportes);
110 scrollArea.setBorder(BorderFactory.createTitledBorder("Información Registrada"));
111 add(scrollArea, BorderLayout.CENTER);
112
113 // --- PANEL INFERIOR: Conversión ---
114 JPanel panelConversion = new JPanel(new FlowLayout(FlowLayout.CENTER, 15, 10));
115 panelConversion.setBorder(BorderFactory.createTitledBorder("Módulo de Conversión de Temperatura"));
116 panelConversion.add(new JLabel("Celsius (°C):"));
117 txtCelsius = new JTextField(8);
118 panelConversion.add(txtCelsius);
119
120 btnFahrenheit = new JButton("Convertir a Fahrenheit");
121 btnKelvin = new JButton("Convertir a Kelvin");
122
123 btnFahrenheit.addActionListener(this); // Asignación directa del evento
124 btnKelvin.addActionListener(this); // Asignación directa del evento
125
126 panelConversion.add(btnFahrenheit);
127 panelConversion.add(btnKelvin);
128
129 add(panelConversion, BorderLayout.SOUTH);
130
131 }
132
133 // Método único para capturar los eventos de todos los botones
134 public void actionPerformed(ActionEvent e) {
135     Object origen = e.getSource();
136
137     if (origen == btnRegistrarCurso) {
138         controlador.registrarCurso(
139             txtCodCurso.getText().trim(),
140             txtNomCurso.getText().trim(),
141             txtTutorCurso.getText().trim()
142         );
143         txtCodCurso.setText("");
144         txtNomCurso.setText("");
145         txtTutorCurso.setText("");
146     }
147 }

```

```

Projects X
> Centro_de_Animales [main t2]
> Curso [main]
  > Source Packages
    > controller
      > AppController.java
    > main
      > main.java
    > model
      > Curso.java
      > CursoModel.java
      > TareaAcademica.java
    > view
      > MainFrame.java [-/A]
  > Test Packages
  > Libraries
  > Test Libraries
> Ejercicio de Practiva completo [n
> fundamentos [main t1]
> poo [master t3 t1]
> SISTEM [master t3 t1]
> Sistema_Estacionamient [Rebasir
> Sistema_Estacionamiento [maste
> Sistema_Notas [main]

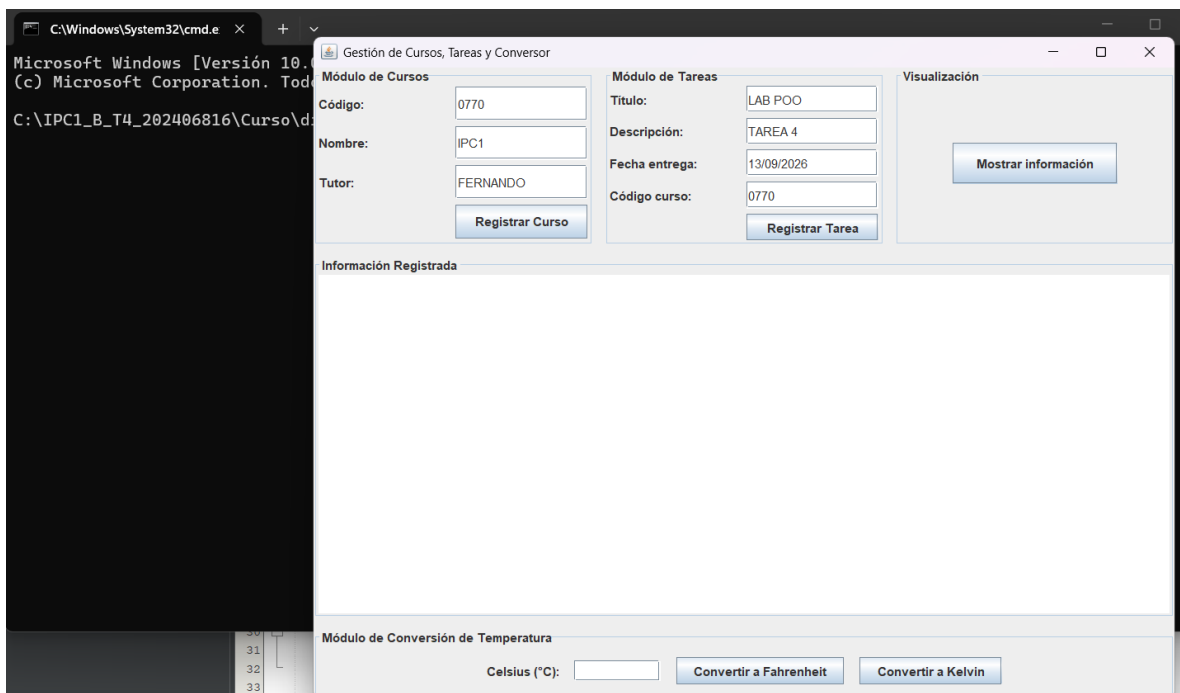
AppController.java X MainFrame.java [-/A] X
Source History
146 } else if (origen == btnRegistrarTarea) {
147     controlador.registrarTarea(
148         txtTituloTarea.getText().trim(),
149         txtDescTarea.getText().trim(),
150         txtFechaTarea.getText().trim(),
151         txtCodCursoTarea.getText().trim()
152     );
153     txtTituloTarea.setText("");
154     txtDescTarea.setText("");
155     txtFechaTarea.setText("");
156     txtCodCursoTarea.setText("");
157
158 } else if (origen == btnMostrarInfo) {
159     mostrarInformacion();
160
161 } else if (origen == btnFahrenheit) {
162     convertirTemperatura(true);
163
164 } else if (origen == btnKelvin) {
165     convertirTemperatura(false);
166 }
167 }
168
169 private void mostrarInformacion() {
170     StringBuilder sb = new StringBuilder();
171
172     sb.append("==== CURSOS =====\n");
173     if (modelo.getCantidadCursos() == 0) {
174         sb.append("No hay cursos registrados.\n");
175     } else {
176         for (int i = 0; i < modelo.getCantidadCursos(); i++) {
177             Curso c = modelo.getCursos()[i];
178             sb.append("Código: ").append(c.getCodigo()).append("\n");
179             sb.append("Nombre: ").append(c.getNombre()).append("\n");
180             sb.append("Tutor: ").append(c.getTutor()).append("\n\n");
181         }
182     }
183 }

```

```
AppController.java x MainFrame.java [-/A] x
Source History
186 sb.append("No hay tareas registradas.\n");
187 } else {
188     for (int i = 0; i < modelo.getCantidadTareas(); i++) {
189         TareaAcademica t = modelo.getTareas()[i];
190         sb.append("Título: ").append(t.getTitulo()).append("\n")
191           .append("Descripción: ").append(t.getDescripcion()).append("\n")
192           .append("Fecha entrega: ").append(t.getFechaEntrega()).append("\n")
193           .append("Código curso asociado: ").append(t.getCodigoCurso()).append("\n\n");
194     }
195 }
196
197 txtAreaReportes.setText(sb.toString());
198 }
199
200 private void convertirTemperatura(boolean esFahrenheit) {
201     String texto = txtCelsius.getText().trim();
202
203     if (texto.isEmpty()) {
204         JOptionPane.showMessageDialog(this, "Ingrese un valor numérico para la temperatura en Celsius.", "Campo Vacío", JOptionPane.ERROR_MESSAGE);
205         return;
206     }
207
208     try {
209         double celsius = Double.parseDouble(texto);
210         if (esFahrenheit) {
211             double fahrenheit = (celsius * 9.0 / 5.0) + 32;
212             JOptionPane.showMessageDialog(this, celsius + " °C equivalen a " + fahrenheit + " °F", "Resultado Conversión", JOptionPane.INFORMATION_MESSAGE);
213         } else {
214             double kelvin = celsius + 273.15;
215             JOptionPane.showMessageDialog(this, celsius + " °C equivalen a " + kelvin + " K", "Resultado Conversión", JOptionPane.INFORMATION_MESSAGE);
216         }
217     } catch (NumberFormatException e) {
218         JOptionPane.showMessageDialog(this, "Ingrese únicamente datos numéricos válidos para la temperatura.", "Error de Formato", JOptionPane.ERROR_MESSAGE);
219     }
220 }
221 }
```

4. PRIMERO DESCARGUE MI JAR ATRAVEZ DE EL COMPROBE EL FUNCIONAMIENTO DE DICHA TAREA

1 Mi primer paso fue llenar la información de los recuadros para ver si me permitía colocar de forma correcta mi información.



2. Segundo paso fue presionar el botón de Registrar Curso como también de Modulo de Tareas y aparece un recuadro de aviso que quedo registrado

Nota: El botón de los registros solo los Guardar no aparecen directamente en el Panel cuando presionamos el botón de Mostrar información es cuando nos aparecerá la información escrita

The screenshot shows the 'Gestión de Cursos, Tareas y Conversor' application. It has three main sections: 'Módulo de Cursos', 'Módulo de Tareas', and 'Visualización'. The 'Módulo de Cursos' section contains input fields for 'Código:' (0770), 'Nombre:' (IPC1), and 'Tutor:' (FERNANDO), with a 'Registrar Curso' button. The 'Módulo de Tareas' section contains input fields for 'Título:' (LAB POO), 'Descripción:' (TAREA 4), 'Fecha entrega:' (13/09/2026), and 'Código curso:' (0770), with a 'Registrar Tarea' button. The 'Visualización' section has a 'Mostrar información' button. Below these sections is the 'Información Registrada' area, which is currently empty. A message box titled 'Message' is displayed in the center, showing an information icon and the text 'Curso registrado correctamente.' with an 'OK' button.

The screenshot shows the 'Gestión de Cursos, Tareas y Conversor' application. The 'Módulo de Cursos' section now has empty input fields for 'Código:', 'Nombre:', and 'Tutor:', with a 'Registrar Curso' button. The 'Módulo de Tareas' section contains input fields for 'Título:' (LAB POO), 'Descripción:' (TAREA 4), 'Fecha entrega:' (13/09/2026), and 'Código curso:' (0770), with a 'Registrar Tarea' button. The 'Visualización' section has a 'Mostrar información' button. Below these sections is the 'Información Registrada' area, which now displays the following text: '==== CURSOS ====', 'Código: 0770', 'Nombre: IPC1', 'Tutor: FERNANDO', '==== TAREAS ====', and 'No hay tareas registradas.' A message box titled 'Message' is displayed in the center, showing an information icon and the text 'Tarea registrada correctamente.' with an 'OK' button. At the bottom of the application is the 'Módulo de Conversión de Temperatura' section, which includes a 'Celsius (°C):' input field and two buttons: 'Convertir a Fahrenheit' and 'Convertir a Kelvin'.

3. A qui podemos apreciar el registro de lo que habíamos escrito en Modulo Cursos nos genera automáticamente la información colocada en esa área

Gestión de Cursos, Tareas y Conversor

Módulo de Cursos

Código:

Nombre:

Tutor:

Registrar Curso

Módulo de Tareas

Título:

Descripción:

Fecha entrega:

Código curso:

Registrar Tarea

Visualización

Mostrar información

Información Registrada

===== CURSOS =====

Código: 0770
Nombre: IPC1
Tutor: FERNANDO

===== TAREAS =====

No hay tareas registradas.

Módulo de Conversión de Temperatura

Celsius (°C):

Convertir a Fahrenheit **Convertir a Kelvin**

4. comprobamos nuevamente que pasa lo mismo en el Modulo de Tareas, cuando presionamos el botón de Mostrar Información automáticamente nos guarda el registro

Gestión de Cursos, Tareas y Conversor

Módulo de Cursos

Código:

Nombre:

Tutor:

Registrar Curso

Módulo de Tareas

Título:

Descripción:

Fecha entrega:

Código curso:

Registrar Tarea

Visualización

Mostrar información

Información Registrada

===== CURSOS =====

Código: 0770
Nombre: IPC1
Tutor: FERNANDO

===== TAREAS =====

Título: LAB POO
Descripción: TAREA 4
Fecha entrega: 13/09/2026
Código curso asociado: 0770

5. Conversión de Celsius a FAHRENHEIT funcionando

Aquí solo debemos de colocar en el espacio en blanco la cantidad que deseamos convertir presionamos la conversión y nos aparece por pantalla una ventanita con el resultado

The screenshot shows the 'Gestión de Cursos, Tareas y Conversor' application. It features three main sections: 'Módulo de Cursos', 'Módulo de Tareas', and 'Visualización'. The 'Módulo de Cursos' section has input fields for 'Código:', 'Nombre:', and 'Tutor:', with a 'Registrar Curso' button below. The 'Módulo de Tareas' section has input fields for 'Título:', 'Descripción:', 'Fecha entrega:', and 'Código curso:', with a 'Registrar Tarea' button below. The 'Visualización' section contains a 'Mostrar información' button. Below these sections is the 'Información Registrada' area, which displays details for a course (Código: 0770, Nombre: IPC1, Tutor: FERNANDO) and a task (Título: LAB POO, Descripción: TAREA 4, Fecha entrega: 13/09/2026, Código curso asociado: 0770). At the bottom is the 'Módulo de Conversión de Temperatura', which includes a 'Celsius (°C):' input field with the value '25', and two buttons: 'Convertir a Fahrenheit' and 'Convertir a Kelvin'. A modal window titled 'Resultado Conversión' is displayed in the center, showing an information icon, the text '25.0 °C equivalen a 77.0 °F', and an 'OK' button.

6. Conversión de Celsius a KELVIN funcionando

Aquí solo debemos de colocar en el espacio en blanco la cantidad que deseamos convertir presionamos la conversión y nos aparece por pantalla una ventanita con el resultado

This screenshot is identical to the one above, showing the same application interface. The 'Módulo de Conversión de Temperatura' section at the bottom has the 'Celsius (°C):' input field set to '25'. The 'Convertir a Fahrenheit' button is highlighted, and the modal window titled 'Resultado Conversión' is displayed in the center, showing an information icon, the text '25.0 °C equivalen a 298.15 K', and an 'OK' button.

CONCLUSION DE MI TAREA FINAL

La implementación de la arquitectura Modelo-Vista-Controlador (MVC) permitió desacoplar la interfaz gráfica de la lógica del sistema, facilitando la validación de datos con JOptionPane y el mantenimiento estructurado del código.

El uso de arreglos nativos y contadores manuales demostró la efectividad de la programación orientada a objetos para administrar relaciones entre entidades (Curso y TareaAcademica) sin depender del framework de colecciones.

La construcción manual de interfaces en Swing mediante distribuidores de diseño (LayoutManagers) brindó un control preciso sobre la organización visual y el manejo de eventos a través de ActionListener.